



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – BANCO DE DADOS 2
PROF. MSc. LUCAS DEBATIN

GUSTAVO HENRIQUE STAHL MÜLLER

RELATÓRIO DE DESENVOLVIMENTO TRABALHO 6

ITAJAÍ
2021

SUMÁRIO

1. INTRODUÇÃO	3
2. ESTRUTURA	3
2.1. SERVIÇO, SOLUÇÃO E ESCOPO	3
2.1.1 TECNOLOGIAS	3
2.2. PROBLEMA	3
3. APLICAÇÃO	4
3.1. FRONT-END	4
3.2. BACK-END	6

1. INTRODUÇÃO

Este relatório demonstra o desenvolvimento do primeiro item do sexto trabalho da disciplina de Banco de Dados 2, englobando aspectos iniciais do trabalho à ser feito e definindo muitas características suas.

2. ESTRUTURA

2.1. SERVIÇO, SOLUÇÃO E ESCOPO

O serviço à ser criado será o de um imageboard, onde usuários podem criar *threads* e *posts*. *Posts* são postagens criadas pelos usuários, e podem conter texto e hipermídia (imagens, vídeos, entre outros). *Threads* remetem à uma coleção de *posts* que compartilham um gênero em comum (como jogos, esportes, notícias, tecnologia, entre outros).

O servidor disponibilizará uma API REST através de webservices que suporta o CRUD de duas entidades: *threads* e *posts*, onde cada *post* deve pertencer à uma *thread* pré-existente. Para o escopo desse trabalho, não será criado nenhum sistema de autenticação ou criação de conta; Logo, ele focará solenemente no armazenamento das entidades em um banco de dados não-relacional e na criação da API.

Um conjunto de páginas web também serão criadas para conceder ao usuário uma interação gráfica com os dados da API, permitindo inserção, leitura, atualização, remoção e inserção tanto para *threads* quanto para *posts*.

Como *threads* e *posts* podem conter conteúdo extremamente variado (pois a solução não terá restrições sobre o que pode ser postado), o público alvo dessa solução é o usuário médio da internet, que busca informação sobre assuntos diversos.

2.1.1 TECNOLOGIAS

- Os dados das *threads* e *posts* serão armazenados pelo MongoDB, em coleções separadas.
- As páginas web que interagem com a API serão feitas em React e MaterialUI.
- O servidor responsável por disponibilizar a API e interagir com o MongoDB será escrito em Node.js (com o framework Express.js).

2.2. PROBLEMA

O problema que o serviço descrito acima procura resolver é a da interação entre usuários pela Internet para assuntos diversos de maneira anônima, tanto para aqueles que buscam informação quanto para aqueles que buscam entretenimento.

A solução também busca organizar *posts* em *threads* que possuem um gênero correspondente; Através desse agrupamento, a procura de informações específicas se torna mais fácil.

3. APLICAÇÃO

3.1. FRONT-END

A interface padrão da aplicação é mostrada na figura abaixo:

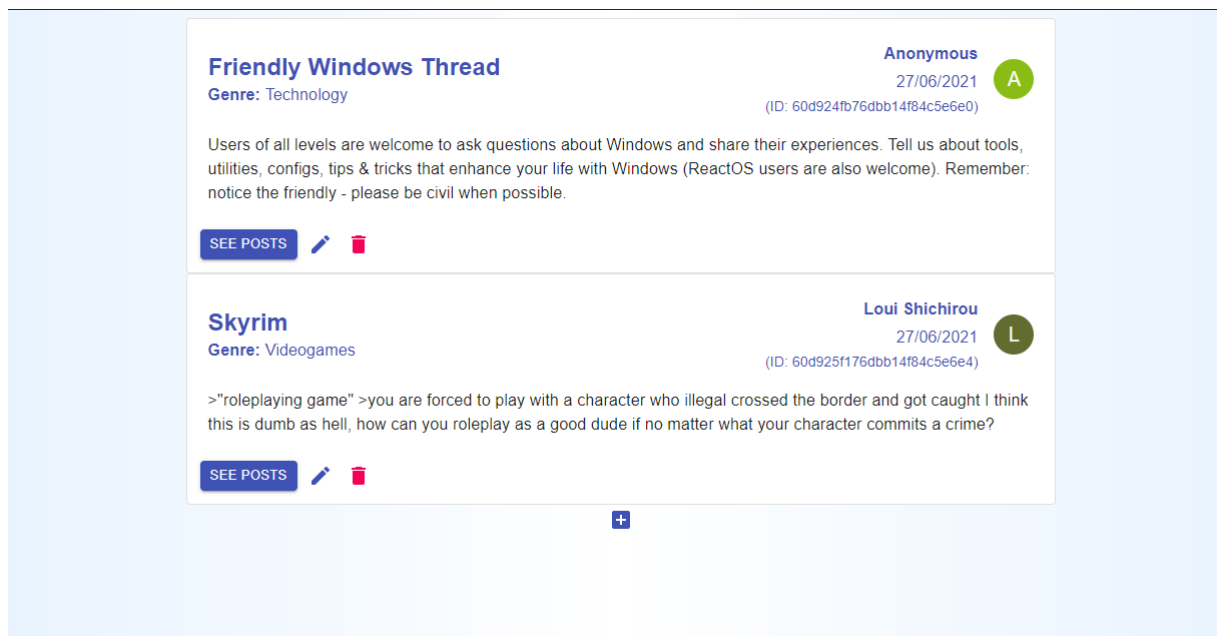


Figura 1: Interface de *threads* com duas entidades. Fonte: O Autor.

Nela, é possível perceber a existência de duas *threads* com os seus respectivos dados. Cada *thread* pode ser editada ou excluída; Basta que o usuário clique no botão de editar ou excluir, respectivamente.

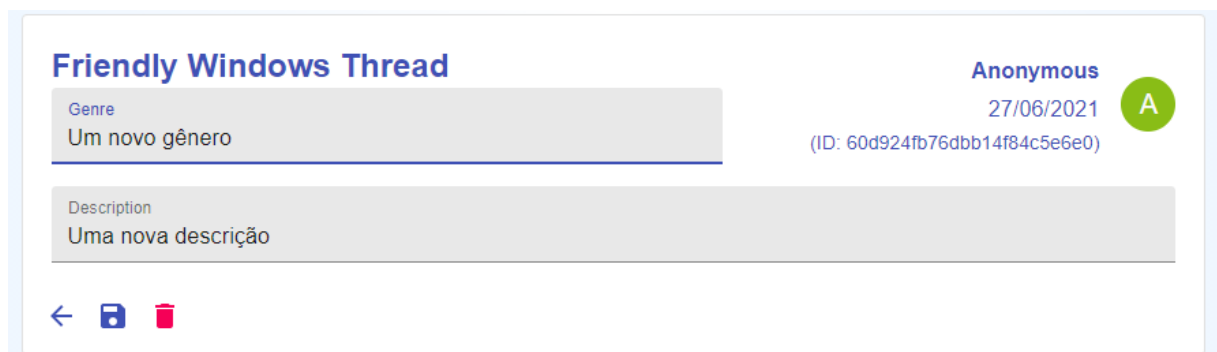


Figura 2: Interface de atualização de *thread*. Fonte: O Autor.

Ao clicar no botão “See Posts”, o usuário será redirecionado à uma outra página que contém os *posts* respectivos à *thread* selecionada.

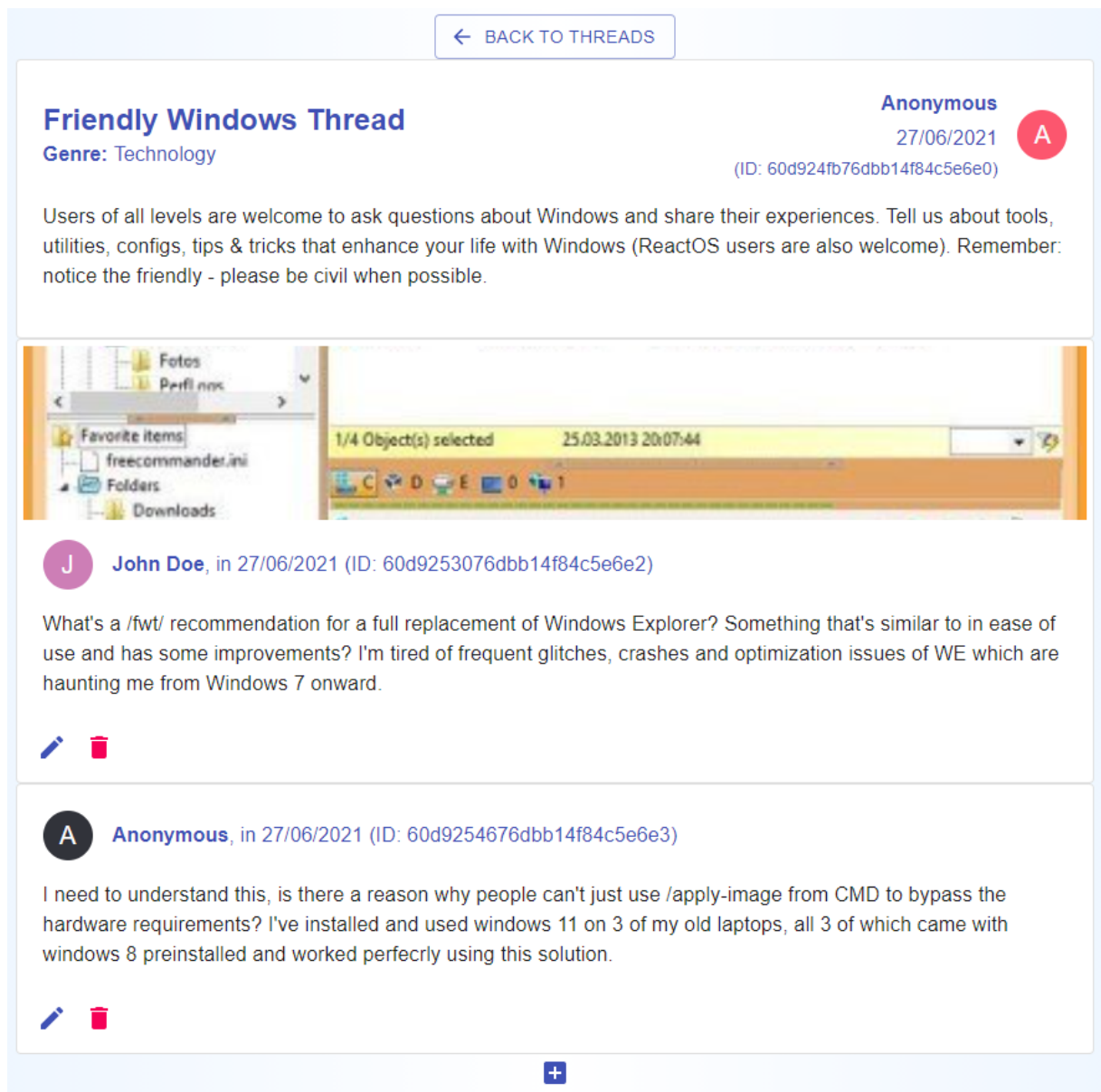


Figura 3: Interface de *posts* de uma certa *thread*. Fonte: O Autor.

O usuário pode voltar para a listagem de *threads* clicando no botão “Back to Threads”. É possível editar os *posts* da mesma maneira que *threads* são editadas:

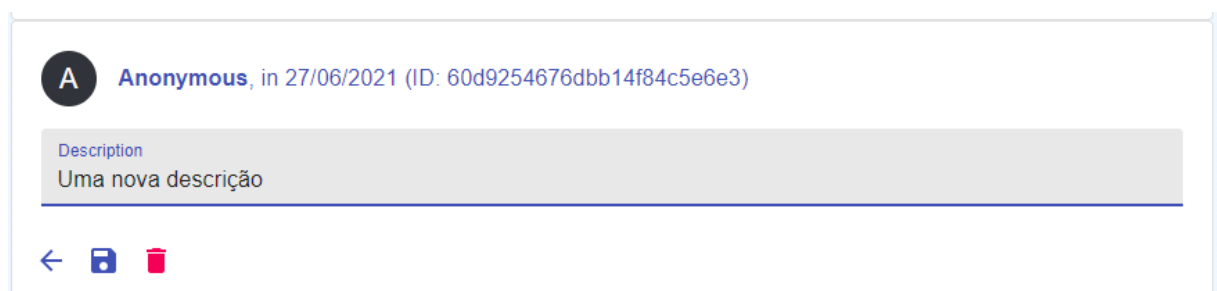


Figura 4: Interface de atualização de *post*. Fonte: O Autor.

3.2. BACK-END

A API do serviço foi desenvolvida em Node.JS, e interage com o MongoDB através de um driver nativo, ou seja, ela não usa bibliotecas intermediárias como o *Mongoose*. Além disso, a API recebe e devolve os dados em formato JSON. Os endpoints fornecidos são os seguintes:

Threads

- GET /thread: Selecionar todas as *threads*.
- GET /thread/:id: Selecionar uma *thread* por ID.
- POST /thread: Criar uma *thread*.
- PUT /thread/:id: Atualizar uma *thread* por ID.
- DELETE /thread/:id: Deletar uma *thread* por ID.

Posts

- GET /post: Selecionar todos os *posts*.
- GET /post/:id: Selecionar um *post* por ID.
- GET /thread/:id/post: Selecionar todos os *posts* de uma *thread* por ID.
- POST /post: Criar um *post*.
- PUT /post/:id: Atualizar um *post* por ID.
- DELETE /thread/:id: Deletar um *post* por ID.

Implementação

- Endpoints de criação utilizam o método `insertOne()`.
- Endpoints de seleção utilizam o método `find()` ou `findOne()`.
- Endpoints de atualização utilizam o método `updateOne()`.
- Endpoints de remoção utilizam o método `deleteMany()`.

Os ID's únicos das threads e posts são criados no momento que elas são inseridas em suas coleções respectivas, sendo que esse ID único é visível na interface por qualquer usuário, como demonstrado anteriormente.